LSTM-ARO 簡報｜第 12–19 頁 研究筆記與講稿

> 依據：新版網頁簡報（共 27 頁）與論文原文 §3.4–§3.6、Eqs. (6)–(20)、Tables 5–6、Fig. 2。
> 建議時長：第 12–19 頁約 9 分鐘（30 分鐘論文報告中的中段）。

一、核心觀念速懂（被問到要能回答）

1. GA 遺傳演算法（第 12 頁）

- 一個「個體」= 一組超參數（染色體）。每一代：評估適應度 → **選擇**較好的 → **交配**交換參數片段 → **突變**隨機改動 → 產生下一代。
- 論文中 GA 與 ARO 一樣跑 50 個個體 × 50 代；但論文**沒有給 GA 的交配率、突變率、選擇方式**。
- GA 與 ARO 的差別只在「候選解怎麼更新」：GA 用選擇 / 交配 / 突變，ARO 用覓食 / 躲藏。

2. ARO 的全貌（第 13 頁）

- 一隻兔子的「位置」= 一組超參數（一個向量，每個維度是一個超參數）。
- **繞路覓食**（探索）：跑到另一隻兔子附近找新區域。**隨機躲藏**（開發）：在自己附近挖洞，細修目前的解。
- 每一代用能量因子 A(t) 決定這隻兔子要覓食還是躲藏；新位置比較好才接受（貪婪接受）。
- ARO 除了兔群大小 N 和迭代次數 T 之外，沒有其他需要人工設定的演算法參數；GA 則還要設定交配率、突變率等。

3. 公式總表（論文 Eqs. 6–20）

式	公式	意思
(6)	$v_i(t+1) = x_j(t) + R\cdot(x_i(t) - x_j(t)) + \text{round}(0.5(0.05 + r_1))\cdot n_1$	繞路覓食：以另一隻兔子 j 為基準移動，再加擾動
(7)	$R = L \cdot c$	移動係數 = 步長 × 維度遮罩
(8)	$L = (e - e^{((t-1)/T)^2}) \cdot \sin(2\pi r_2)$	步長：上限從約 1.72 縮到接近 0，sin 決定方向
(9)(10)	$c(k) = 1$ 若 k 是隨機選中的維度，否則 0；g = randperm(d)	只讓部分維度移動
(11)	$n_1 \sim N(0,1)$	擾動用的標準常態隨機數
(12)	$b_{i,j}(t) = x_i(t) + H \cdot g \cdot x_i(t)$	在自己周圍、第 j 個維度挖一個洞
(13)	$H = ((T - t + 1) / T) \cdot r_4$	挖洞範圍，隨迭代從約 1 線性縮小到 1/T（再乘隨機數 r_4）
(16)	$v_i(t+1) = x_i(t) + R\cdot(r_4 \cdot b_{i,r}(t) - x_i(t))$	隨機躲藏：從洞穴中隨機選一個 $b_{i,r}$，朝它移動
(19)	$x_i(t+1) = v_i(t+1)$，若 $f(v_i(t+1))$ 比 $f(x_i(t))$ 好；否則維持 $x_i(t)$	貪婪接受
(20)	$A(t) = 4 \cdot (1 - t/T) \cdot \ln(1/r)$	能量因子：A > 1 覓食，A ≤ 1 躲藏

易混淆：Eq. (6) 的 j 是「另一隻兔子」的編號；Eq. (12) 的 j 是「維度」的編號。

4. 步長 L 與能量 A 的直覺（第 15、17 頁）

迭代進度 t/T	L 的上限 $e - e^{((t-1)/T)^2}$	A > 1（覓食）的機率
開始（t = 1）	1.72	約 78%
一半	約 1.43	約 61%
四分之三	約 0.97	約 37%
結束（t = T = 50）	約 0.11	0%

A > 1 的機率 = $\exp(-1 / (4(1 - t/T)))$，由 r 為 0–1 均勻分布推得。結論：前期多半大步探索，後期多半在附近細修。

5. 搜尋空間（第 18 頁，論文 Tables 5–6）

- 9 個維度：x_0 第一層神經元（1–20）、x_1 / x_3 第二 / 三層是否存在（0 或 1）、x_2 / x_4 第二 / 三層神經元（1–20）、x_5 Dense 神經元（1–20）、x_6 Dropout（0.3–0.7，5 種）、優化器（5 種）、學習率（5 種）。
- 組合數約 20 × 21 × 21 × 20 × 5 × 5 × 5 ≈ **2,205 萬種**（第二、三層各有「不存在」或 1–20 個神經元，共 21 種）；ARO 只評估 2,500 組，約 0.01%。

- 論文沒有說明：兔子的位置是連續值，如何對應到整數神經元數和離散選項（常見做法是四捨五入或換成索引）。

6. 完整流程（第 19 頁，論文 Fig. 2）

初始化兔群（隨機超參數）→ 依每隻兔子建 LSTM → 用訓練資料訓練、用測試資料算誤差作為適應度 → 依 A(t) 覓食或躲藏產生候選位置 → 誤差較低才接受 → 重複到第 50 代 → 輸出最佳解。

二、講稿（約 9 分鐘）

Slide 12 | GA 遺傳演算法（約 1 分鐘）

在介紹 ARO 之前，先看比較組 LSTM-GA 用的遺傳演算法。GA 把一組超參數當成一條染色體，一開始隨機產生一群。每一代先訓練模型、計算誤差當作適應度，再用選擇留下表現好的，用交配交換兩組參數的片段，用突變隨機改動一部分，產生下一代。GA 和 ARO 都在模型外面搜尋超參數，差別只在候選解怎麼更新：GA 是選擇、交配、突變，ARO 則是覓食和躲藏。

Slide 13 | ARO 在模仿什麼（約 1 分鐘）

ARO 是 2022 年提出的演算法，模仿兔子的兩種求生策略。第一種是繞路覓食：兔子不吃自己窩邊的草，會跑到別的地方覓食，對應到演算法就是參考其他兔子的位置、加上擾動，去搜尋新的區域，這是探索。第二種是隨機躲藏：兔子會在窩的周圍挖好幾個洞，隨機挑一個躲，對應到演算法就是在自己附近產生候選位置，把找到的好區域挖深，這是開發。在這篇論文裡，每隻兔子的位置就是一組 LSTM 超參數。

Slide 14 | 繞路覓食（約 1 分 20 秒）

這是繞路覓食的公式，也就是論文的式 6。第 i 隻兔子下一代的候選位置 v_i，是以隨機選出的另一隻兔子 x_j 為基準，加上 R 乘以兩隻兔子的位置差，再加上一個隨機擾動 q 乘 $n_ı$。這裡 N 是兔群大小、T 是最大迭代次數，論文都設為 50。R 是移動係數，下一頁說明；q 由隨機數 $r_ı$ 決定是 0 還是 1，決定這次要不要加擾動；$n_ı$ 是標準常態分布的隨機數。擾動的目的是幫助跳出局部最佳解。

Slide 15 | 步長、維度與接受條件（約 1 分 10 秒）

移動係數 R 等於步長 L 乘上 c。c 是一個只有 0 和 1 的遮罩，只讓隨機選中的部分維度移動，也就是一次只改幾個超參數。步長 L 由兩部分相乘：前半段是步伐上限，一開始大約 1.72，隨著迭代縮小到接近 0；後半段 $\sin(2\pi r_2)$ 是 −1 到 1 的隨機數，決定方向和用掉上限的幾成。所以前期步伐大、廣泛探索，後期步伐小、細部調整。最後是貪婪接受：只有新位置的適應度比較好，兔子才會移動，否則留在原地。要強調的是，ARO 更新的是超參數，不是 LSTM 的權重。

Slide 16 | 隨機躲藏（約 1 分 10 秒）

第二種行為是隨機躲藏。兔子會在自己目前的位置附近、每個維度各挖一個洞，就是式 12 的 $b_{i,j}$。挖洞的範圍由 H 控制，H 從大約 1 隨迭代線性縮小，所以前期洞挖得比較遠、後期比較近。接著用式 16，從這些洞裡隨機選一個，朝它移動，得到候選位置 v_i。同樣地，候選位置要比原本好才會被接受。

Slide 17 | 什麼時候覓食或躲藏（約 1 分鐘）

那兔子什麼時候覓食、什麼時候躲藏？由能量因子 A(t) 決定，也就是式 20：A 等於 4 乘以 1 減 t/T，再乘以 ln(1/r)。A 大於 1 就繞路覓食、做全域探索；A 小於等於 1 就隨機躲藏、做局部開發。因為 1 − t/T 會隨迭代變小，A 也跟著變小：一開始大約有八成的機會覓食，到四分之三的時候只剩不到四成，最後一代就一定是躲藏。所以 ARO 會自然地從探索過渡到開發。

Slide 18 | 搜尋空間（約 1 分鐘）

這是 ARO 實際搜尋的內容，也就是論文的 Table 6。第一層 LSTM 的神經元 x_0 是 1 到 20；x_1、x_3 決定第二、第三層存不存在，x_2、x_4 是這兩層的神經元數；x_5 是 Dense 層神經元，x_6 是 Dropout，從 0.3 到 0.7；另外還有五種優化器和五種學習率。整體結構固定是：輸入 20 天，接 1 到 3 層 LSTM，再接 Dense、Dropout，最後輸出 1 個值。粗算這個空間大約有兩千兩百萬種組合，ARO 只評估了其中 2,500 組。

Slide 19 | 完整流程（約 1 分鐘）

最後把整個 LSTM-ARO 串起來，對應論文的 Fig. 2。第一步隨機初始化兔群，每隻兔子是一組超參數；第二步依照每隻兔子的設定建構 LSTM；第三步訓練模型、計算誤差作為適應度；第四步依能量因子決定覓食或躲藏，更新兔子的位置，誤差較低才接受；重複 50 代之後，第五步保留最佳解，得到最終模型。接下來由下一位同學說明實驗結果。

三、這段可能被問的問題

Q1：ARO 和 GA 最大的差別是什麼？ 兩者都是群體搜尋，差在候選解的更新方式：GA 靠選擇、交配、突變；ARO 靠覓食（參考別隻兔子）和躲藏（在自己附近細修），並用能量因子自動從探索轉向開發。ARO 也不需要設定交配率、突變率這類參數。

Q2：兔子的位置是連續值，神經元數和優化器卻是離散的，怎麼對應？ 論文沒有說明。常見做法是把連續值四捨五入成整數，或把範圍切成區間對應到選項索引。

Q3：什麼時候開始以躲藏為主？ A > 1 的機率是 exp(−1/(4(1 − t/T)))：開始時約 78%，一半時約 61%，四分之三時約 37%。所以大約在迭代進行到 60% 左右之後，躲藏（開發）就變成多數。

Q4：為什麼要用貪婪接受？不會卡在局部最佳解嗎？ 貪婪接受保證每隻兔子不會變差；卡住的風險則靠覓食時參考其他兔子、Eq. (6) 的隨機擾動，以及步長裡的隨機方向來降低。

Q5：只評估 2,500 組，相對約 2,205 萬種組合，夠嗎？ 群體最佳化不是窮舉，而是用前面的評估結果引導搜尋方向，所以不需要試遍所有組合；但也因此不能保證找到全域最佳解。另外，每評估一組都要完整訓練一次 LSTM，這也是搜尋要花 8.7 天的原因。

Q6：Eq. (6) 和 Eq. (12) 都有 j，意思一樣嗎？ 不一樣。Eq. (6) 的 j 是隨機選出的另一隻兔子；Eq. (12) 的 j 是搜尋空間的第 j 個維度。

參考文獻

Gülmez, B. (2023). Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm. *Expert Systems with Applications*, 227, 120346. https://doi.org/10.1016/j.eswa.2023.120346

Wang, L., Cao, Q., Zhang, Z., Mirjalili, S., & Zhao, W. (2022). Artificial rabbits optimization: A new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. *Engineering Applications of Artificial Intelligence*, 114, 105082.